



Welcome to

AASD 4011: Deep Learning II

Credit hours: 48

Instructor: Dr. Moe Fadaee

Email: mo.fadaee@georgebrown.ca

George Brown College

Agenda:

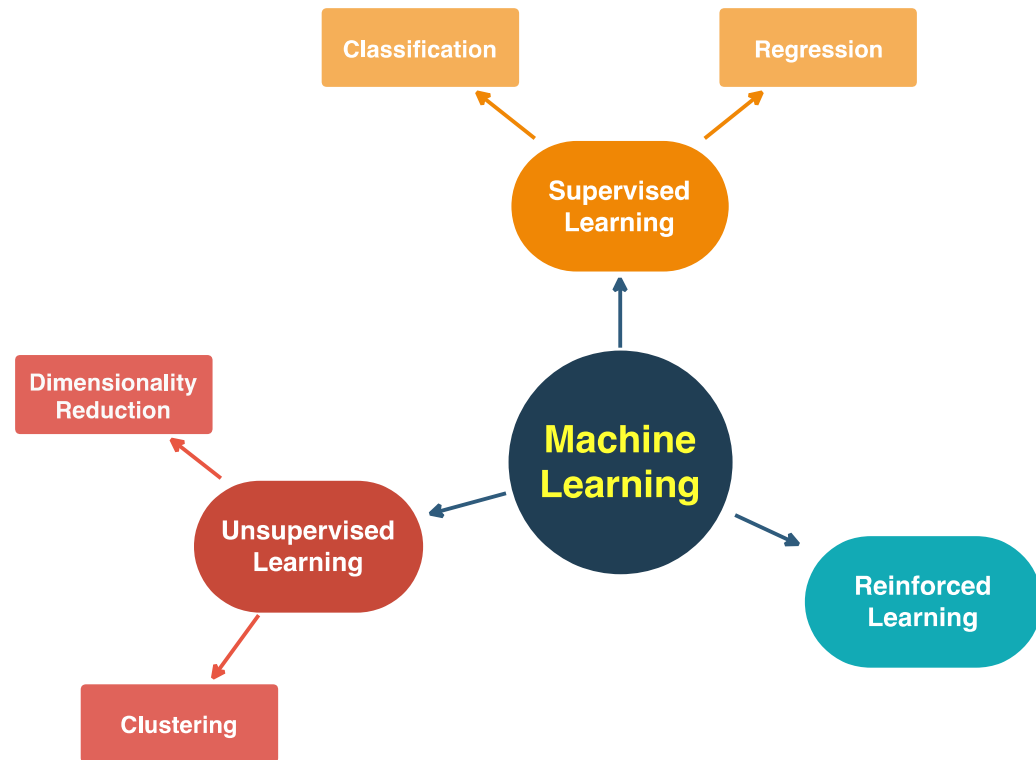
1. OpenCV library and important functionality
2. Labeling tools

Machine Learning subcategories:

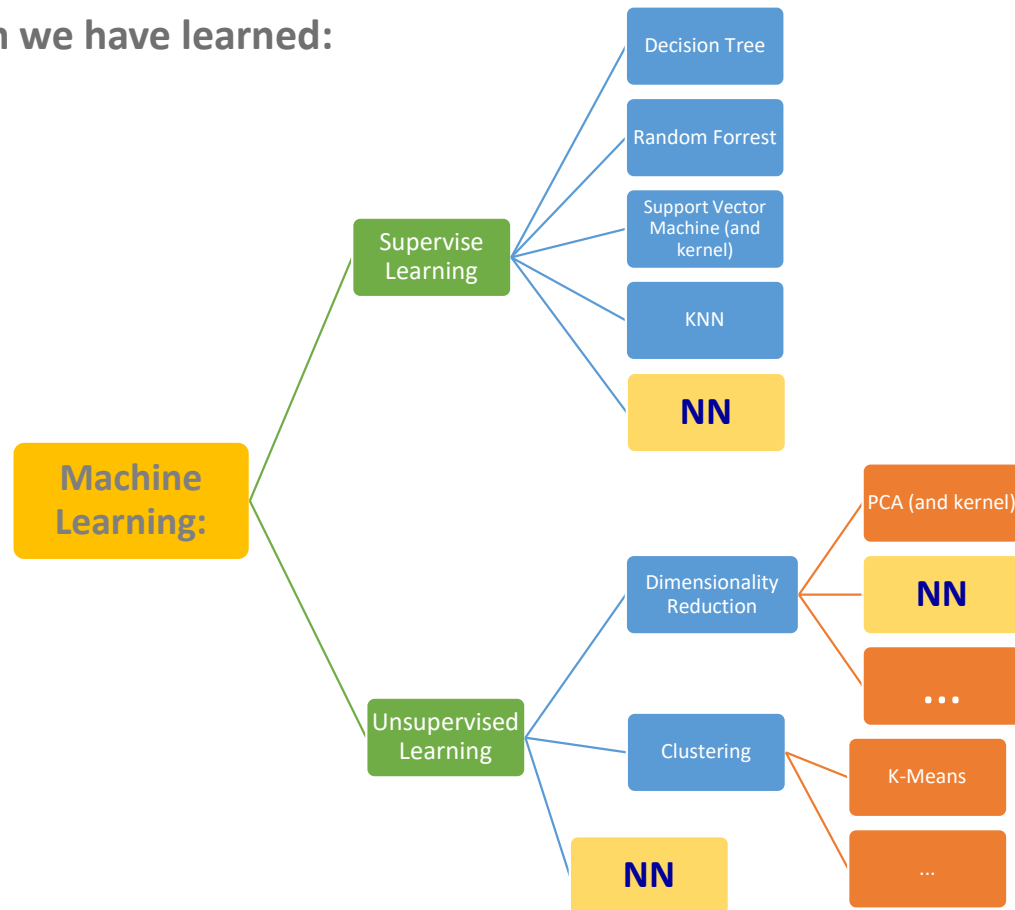
- I. Supervised Learning
- II. Un-supervised Learning
- III. Reinforced learning
- IV. Others...

Where is NN:

Almost all of them



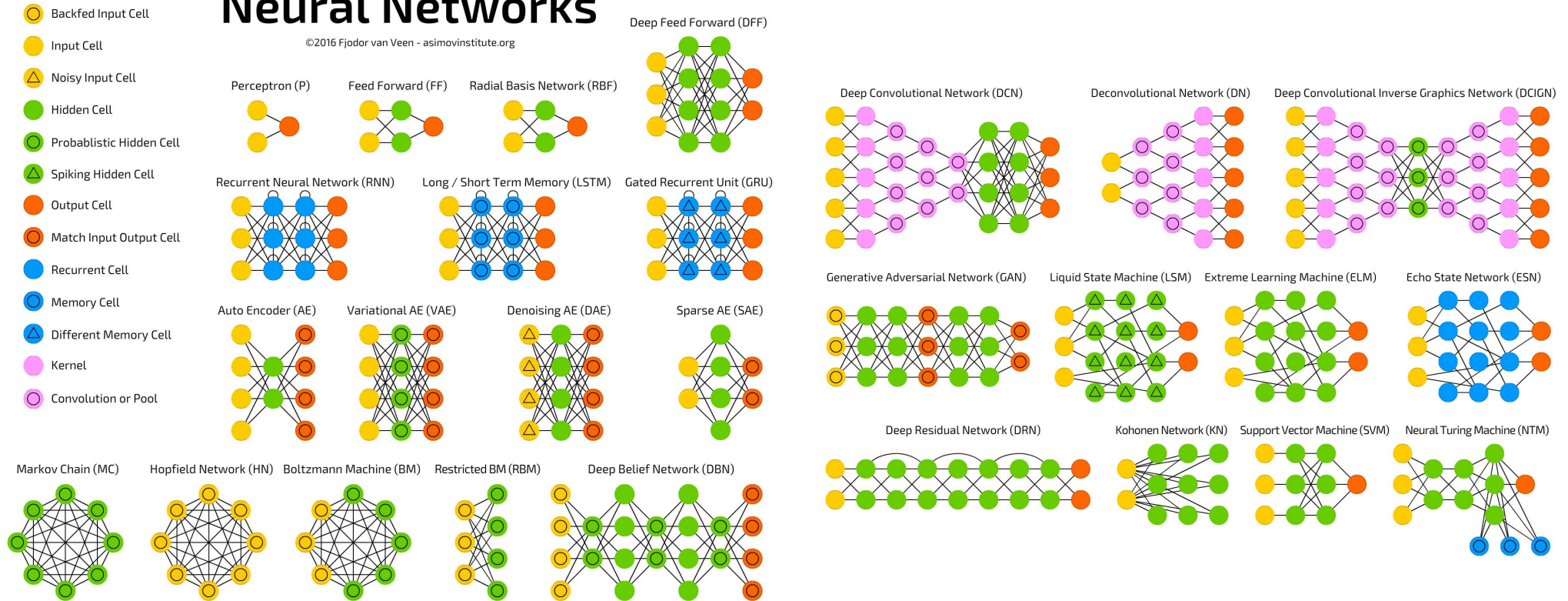
What algorithm we have learned:



Different structures of Neural Networks:

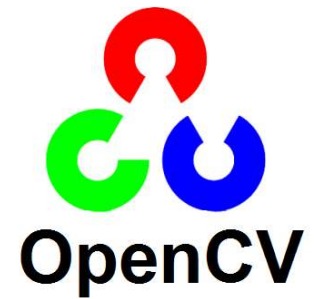
A mostly complete chart of Neural Networks

©2016 Fjodor van Veen - asimovinstitute.org



OpenCV library:

OpenCV is a library with so many functionality build for real time computer vision. The library initially developed in 1999 by intel.



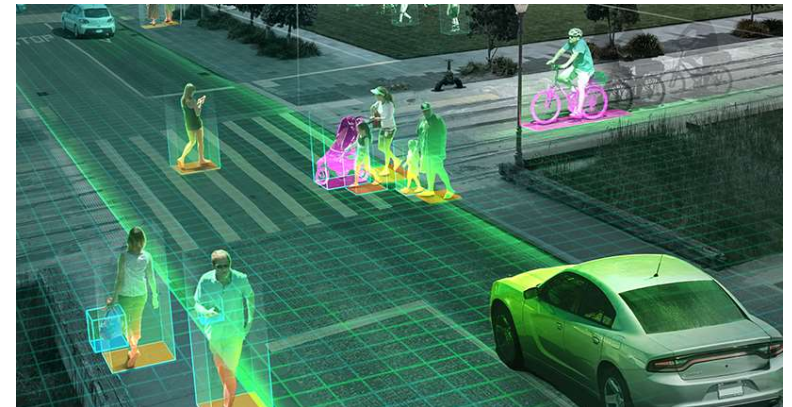
Application of opencv include but not limited to:

- 2D and 3D feature toolkits
- Object detection
- Segmentation and recognition
- Facial recognition system
- Gesture recognition
- Human–computer interaction (HCI)
- Mobile robotics
- Motion understanding
- ...

<https://opencv.org>

<https://github.com/opencv/opencv>

Installation guide: <https://learnopencv.com/install-opencv3-on-windows/>



OpenCV library:

Some of main Opencv functionalities:

<https://www.analyticsvidhya.com/blog/2019/03/opencv-functions-computer-vision-python/>

cvlib library:

cvlib is a very easy to use, high level package to build object detection applications.

If you have the following you can install cvlib through pip

Opencv

Tensorflow

For installation please see:

Tensorflow: <https://www.tensorflow.org/install>

Cvlib: <https://github.com/arunponnusamy/cvlib>

Object detection with cvlib:

Face detection

Object detection

The package is capable of detecting 80 categories of common objects, categories are listed here:

<https://github.com/arunponnusamy/object-detection-opencv/blob/master/yolov3.txt>

Feature matching:

https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_feature2d/py_matcher/py_matcher.html

Template matching:

https://docs.opencv.org/3.4/de/da9/tutorial_template_matching.html

Installation:

Please make sure that you have the following packages/tools installed:

CVAT (or other image annotation tool)

Tensorflow object detection api

Image annotation:

Image annotation is referred to the task of annotating images with labels that have been established by the AI engineer. The labels depends on the specific application and task that need to performed.



Image annotation:

Best practices,

Completeness:

Make sure you label all object of interest in the image

Tight bounding box:

Make sure the bounding box is not so wide

Make sure the bounding box cover all of the object and it is not passing through the object

Be consistent

Be consistent in your annotation across all of the images



Image annotation:

Tools,

There are several free and paid tools available, some provide labeling services as well.

Key element:

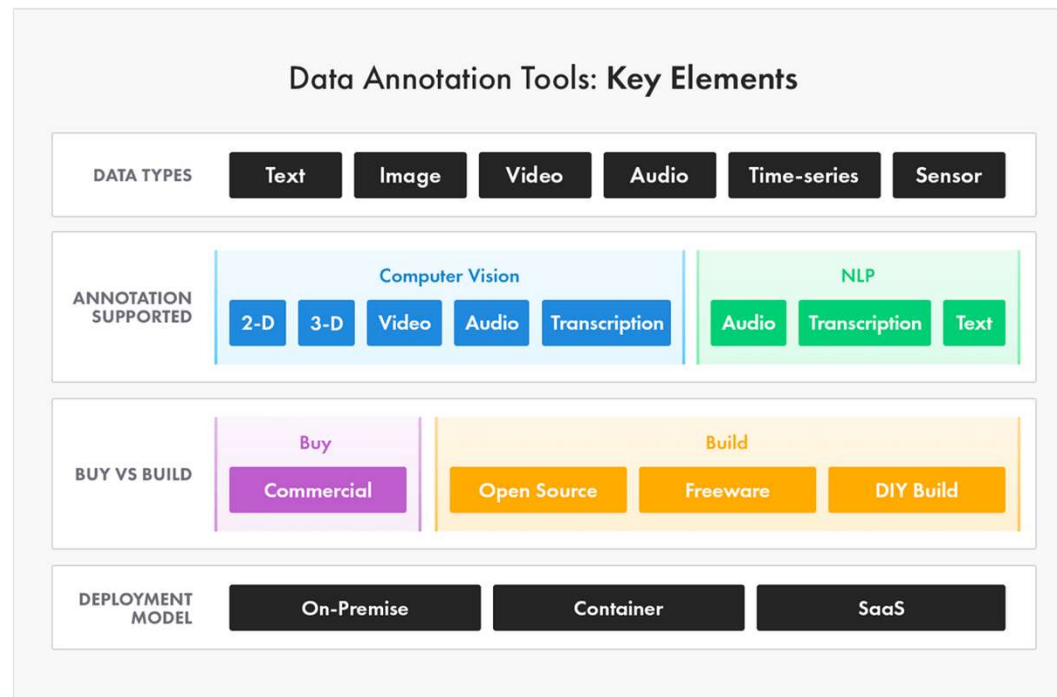


Image annotation:

Tools,

Tool features:

1. Data management
2. Security
3. Annotating tools and methods
 - Images and videos, 2D/3D, box and polygon...
4. Ease of use and flexibility

Image annotation:

Tools,

List of some free tools:

1. Labellmg:
 - <https://github.com/tzutalin/labellmg>
2. CVAT (Computer Vision Annotation Tool)
 - <https://github.com/openvinotoolkit/cvat>
 - <https://www.youtube.com/watch?v=BKLyHOEACFw>
3. VoTT (Visual Object Tagging Tool)
 - <https://github.com/microsoft/VoTT>
4. Labelme
 - <https://github.com/wkentaro/labelme>

More: <https://www.folio3.ai/blog/labelling-images-annotation-tool/>

Image annotation:

Tools,

After annotation convert the individual xml files to a single csv, tools:

<https://github.com/michaelruocco/xml-to-csv>

<https://github.com/ITAgnesmeyer/XmlToCsv>

<https://github.com/andybrodie/xml2csv>

https://github.com/qdraw/tensorflow-face-object-detector-tutorial/blob/master/003_xml-to-csv.py

Image annotation:

Create a label map,

In the next step, you need to create a label map when using the tensorflow object detection.

The label map looks like:

```
item {  
  id: 1  
  name: 'Class_1'  
}  
item {  
  id: 2  
  name: 'Class_2'  
}  
item {  
  id: 2  
  name: 'Class_3'  
}  
...
```

Image annotation: tfrecords,

Finally you need to create your tfrecords to be read by tensorflow.

tfrecords is a simple/efficient format for storing data. The advantages of having your data in this format are:

More efficient storage: less storage.

Fast I/O: Faster I/O, with Parallel I/O capabilities.

Self-contained files: the tfRecord data can be read from a single source.

More on tfrecords:

https://www.tensorflow.org/tutorials/load_data/tfrecord

https://keras.io/examples/keras_recipes/creating_tfrecords/

Several other open source tools are available to convert your data to tfrecords:

https://github.com/abdelrahman-gaber/tf2-object-detection-api-tutorial/blob/master/data_gen/generate_tfrecord.py

https://github.com/DarshanDeshpande/tfrecord-generator/blob/main/image/image_to_tfrecords.py

https://github.com/datitran/raccoon_dataset/blob/master/generate_tfrecord.py

Where to get images:

Google images

Download images from google:

Use <https://github.com/hardikvasa/google-images-download> to download several images at once

Installation: <https://google-images-download.readthedocs.io/en/latest/installation.html>

Using pip: `pip install google_images_download`

Or use other tools like chrome extension called: **Download All Images**

Open images dataset:

<https://storage.googleapis.com/openimages/web/index.html>

Other resources: Kaggle, Roboflow...